Lecture 9. Interactive Proof & Decision Tree

Spring 2026, *Theory of Computation*, IIIS, Tsinghua

Proof systems

- given language L, goal is to prove $x \in L$

- **proof system** for L is a verification algorithm V
  - completeness: $x \in L \Rightarrow \exists$ proof, V accepts (x, proof)

    "true assertions have proofs"
  - soundness: $x \notin L \Rightarrow \forall$ proof*, V rejects (x, proof*)

    "false assertions have no proofs"
  - efficiency: For all x and proof, V(x, proof) runs in polynomial time in |x|

    (so |proof| is poly in |x|)



Classical Proofs

- previous definition:

"classical" proof system

- recall:

$L \in \mathbf{NP}$ iff expressible as

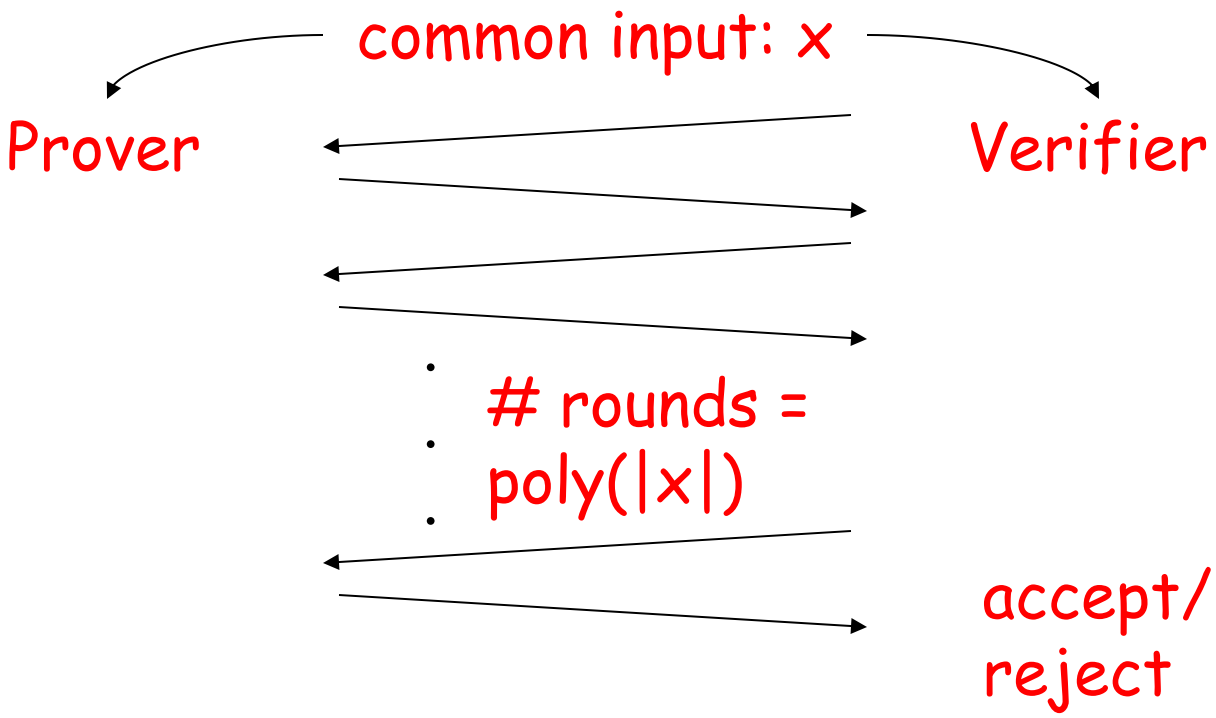
$L = \{ x \mid \exists y, |y| < |x|^k, (x, y) \in R \}$ and $R \in \mathbf{P}$

- **NP** is the set of languages with classical proof systems (R is the verifier)



Interactive Proofs

- **interactive proof system** for L is an interactive protocol (P,V)





Interactive Proof

- Prover P: all-powerful machine



- Verifier V: polynomial time machine
  - cannot compute by itself, so need a prover to convince itself





Interactive Proof

- Prover P: all-powerful machine
- Verifier V: polynomial time TM

- the sequence of interaction
  - input: x
  - $a_1=V(x)$
  - $a_2=P(x,a_1)$
  - …
  - $a_k=V(x,a_1,\ldots,a_{k-1})$

polynomial rounds of interaction, polynomial length of every a_i

- $\langle V,P\rangle(x)=a_k$ is the output of the verifier at the end of the interaction



dIP

- dIP (deterministic interactive proof):
  - completeness: $x \in L \Rightarrow \exists\ P\ \langle V, P\rangle(x)=1$
  - soundness: $x \notin L \Rightarrow \forall\ P^*\ \langle V, P^*\rangle(x)=0$

  - polynomial rounds
  - V: polynomial time TM

- What's the power of dIP?



dIP

- dIP (deterministic interactive proof):
  - completeness: $x \in L \Rightarrow \exists\ P\ \langle V, P\rangle(x)=1$
  - soundness: $x \notin L \Rightarrow \forall\ P^*\ \langle V, P^*\rangle(x)=0$

- dIP=NP



dIP

- dIP (deterministic interactive proof):
  - completeness: $x \in L \Rightarrow \exists P \langle V, P\rangle(x)=1$
  - soundness: $x \notin L \Rightarrow \forall P^* \langle V, P^*\rangle(x)=0$

- dIP=NP
  - $a_1=V(x)$
  - $a_2=P(x,a_1)$
  - …
  - $a_k=V(x,a_1,\ldots,a_{k-1})$

  - the transcript $a_1,\ldots,a_{k-1}$ is the certificate for $x \in L$
  - We just need to check whether $V(x)=a_1$, $V(x,a_1,a_2)=a_3$, …, and $V(x,a_1,\ldots,a_{k-1})=1$



Interactive Proof

- Two new ingredients:
  - **randomness**: verifier tosses coins, errors with some small probability
  - **interaction**: rather than "reading" proof, verifier **interacts** with computationally unbounded **prover**

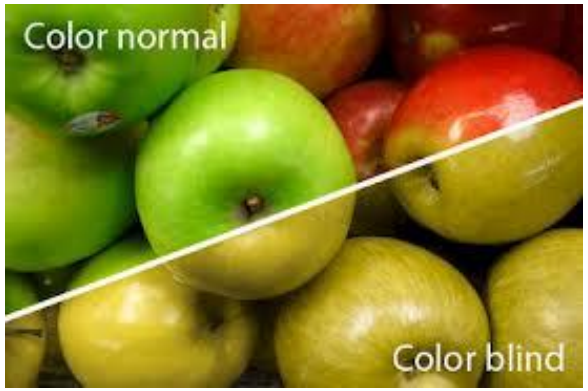
- **NP** proof systems lie in this framework: prover sends proof, verifier does not use randomness



A simple example

- Marla has one red apple and one green apple
- Arthur is color-blind, so they are the same in his eyes

- How can Marla convince Arthur that they are different?





- Marla gives the two apples to Arthur
- Arthur holds the red in left hand and green in right hand

- Arthur turns back and tosses a coin
  - if the coin is "head", keeps the same
  - if the coin is "tail", switches them
- Ask Marla whether he has switched the two apples

•If the two apples are different, Marla always gives the correct answer
•If the two apples are the same, the prob. for correct answer is ½
•So we can repeat to reduce the prob. of error



- In this example, Arthur (the verifier) use randomness to get the "computational" power of Marla that he doesn't have

- Crucial: the verifier holds all the random bits, never show them to the prover!
  - but only in this example?



- In this example, Arthur (the verifier) use randomness to get the "computational" power of Marla that he doesn't have

- Crucial: the verifier holds all the random bits, never show them to the prover!

input: x, random variable **r** $\in \{0,1\}^m$
$a_1 = V(x,\mathbf{r})$
$a_2 = P(x,a_1)$
$a_3 = V(x,\mathbf{r}, a_1,a_2)$
$a_4 = P(x,a_1,a_2,a_3)$
…
$a_k = V(x,\mathbf{r}, a_1,\ldots,a_{k-1})$



Interactive Proofs

- **interactive proof system** for L is an interactive protocol (P,V)
  - completeness: $x \in L \Rightarrow \exists P$

    $Pr_r[V \text{ accepts in } (P,V)(x)] \geq 2/3$
  - soundness: $x \notin L \Rightarrow \forall P^*$

    $Pr_r[V \text{ accepts in } (P^*,V)(x)] \leq 1/3$
  - efficiency: V is p.p.t. machine based on random variable r
  - P cannot see r
- repetition: can reduce error to any ε

- *"Private coins model"*



Interactive Proofs

IP = {L : L has an interactive proof system}

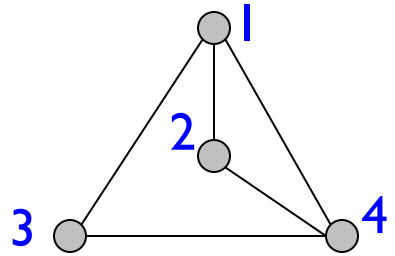
- Observations/questions:
  - philosophically interesting: captures more broadly what it means to be convinced a statement is true
  - clearly **NP** ⊆ **IP.** Potentially larger. How much larger?
  - if larger, randomness is essential

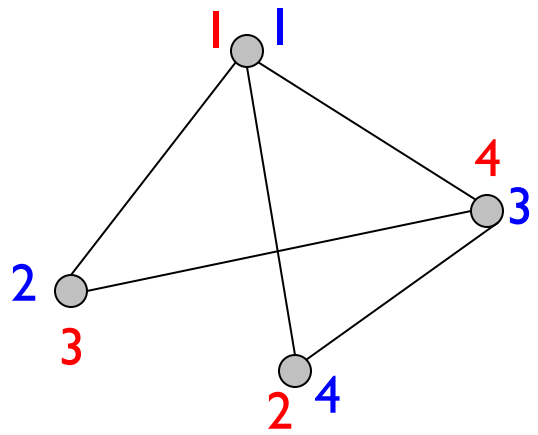


Graph Isomorphism

- graphs $G_0 = (V, E_0)$ and $G_1 = (V, E_1)$ are isomorphic ($G_0 \cong G_1$) if exists a permutation $\pi: V \rightarrow V$ for which

$$(x, y) \in E_0 \Leftrightarrow (\pi(x), \pi(y)) \in E_1$$







Graph Isomorphism

- GI = $\{(G_0, G_1) : G_0 \cong G_1\}$
  - in **NP**
  - not known to be in **P**, or **NP**-complete
- GNI = complement of GI
  - not known to be in **NP**

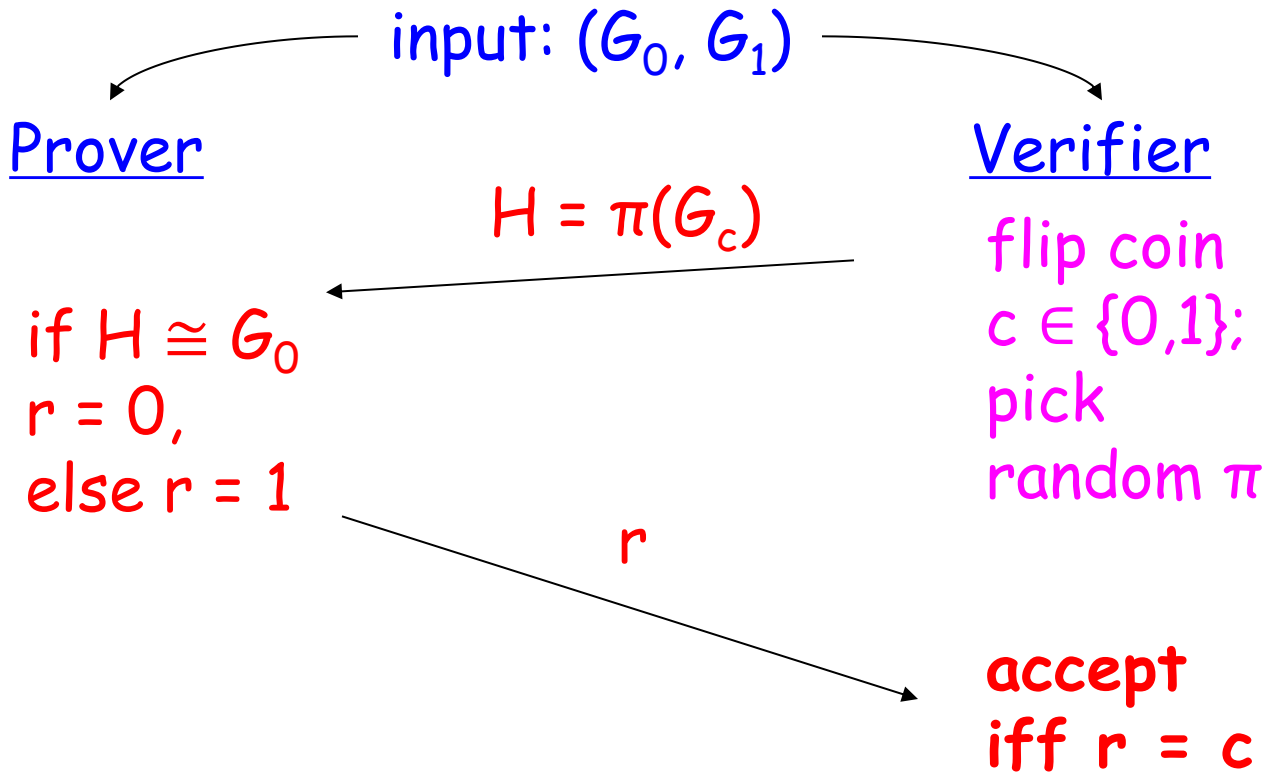
Theorem (GMW): GNI $\in$ **IP**

- indicates **IP** may be more powerful than **NP**



GNI in **IP**

- interactive proof system for GNI:



GNI in **IP**

- completeness:
  - if G_0 is not isomorphic to G_1 then H is isomorphic to exactly one of (G_0, G_1)
  - prover can choose correct r
- soundness:
  - if $G_0 \cong G_1$ then prover sees the same distribution on H for c = 0, c = 1
  - no information on c ⇒ *any* prover P* can succeed with probability at most 1/2



The power of **IP**

- We showed GNI ∈ **IP**
- GNI ∈ **IP** suggests **IP** *(maybe)* more powerful than **NP**, since we don't know how to show GNI in **NP**
- GNI in **coNP**

Theorem (LFKN): **coNP** ⊆ **IP**



The power of **IP**

- We showed GNI ∈ **IP**
- GNI ∈ **IP** suggests **IP** *(maybe)* more powerful than **NP**, since we don't know how to show GNI in **NP**
- GNI in **coNP**

Theorem (LFKN): **coNP** ⊆ **IP**

We prove a generalized version:

#SAT = { (ϕ, k): CNF ϕ has exactly k satisfying assignments} is in **IP**



The power of **IP**

- Proof idea: input: $\phi(x_1, x_2, \ldots, x_n)$
  - prover: "I claim φ has k satisfying assignments"
  - true iff
    - $\phi(0, x_2, \ldots, x_n)$ has k_0 satisfying assignments
    - $\phi(1, x_2, \ldots, x_n)$ has k_1 satisfying assignments
    - $k = k_0 + k_1$
  - prover sends k_0, k_1
  - verifier sends random $c \in \{0,1\}$
  - prover recursively proves "$\phi' = \phi(c, x_2, \ldots, x_n)$ has k_c satisfying assignments"
  - at end, verifier can check for itself



The power of **IP**

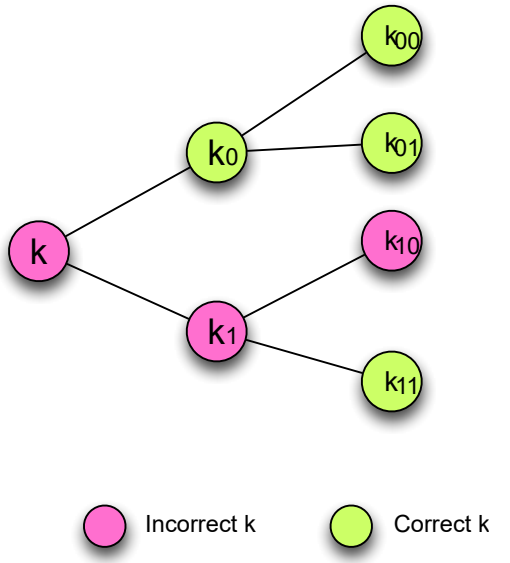
- Analysis of proof idea:
  - Completeness: $\phi(x_1, x_2, \ldots, x_n)$ has k satisfying assignments $\Rightarrow$ accept with prob. 1
  - Soundness: $\phi(x_1, x_2, \ldots, x_n)$ does not have k satisfying assigns. $\Rightarrow$ accept prob. $\leq 1 - 2^{-n}$



The power of **IP**

- Analysis of proof idea:
  - Completeness: $\phi(x_1, x_2, \ldots, x_n)$ has k satisfying assignments $\Rightarrow$ accept with prob. 1
  - Soundness: $\phi(x_1, x_2, \ldots, x_n)$ does not have k satisfying assigns. $\Rightarrow$ accept prob. $\leq 1 - 2^{-n}$

  - Why? It is possible that k is only off by one; verifier only catches the error if coin flips c are successive bits of this assignment





The power of **IP**

- Solution to problem (ideas):
  - replace $\{0,1\}^n$ with $(GF(q))^n$
  - verifier substitutes random field element at each step
  - *vast majority* of field elements catch cheating prover (rather than just 1)

Theorem: #SAT = { (ϕ, k): CNF ϕ has exactly k satisfying assignments} is in **IP**



The power of **IP**

- First step: **arithmetization**
  - transform $\phi(x_1, \ldots x_n)$ into polynomial $p_\phi(x_1, x_2, \ldots x_n)$ of degree d over a field GF(q); q prime > 2^n
  - recursively:
    - $x_i \rightarrow x_i$
    - $\sim\phi \rightarrow (1 - p_\phi)$
    - $\phi \wedge \phi' \rightarrow \quad (p_\phi)(p_{\phi'})$
    - $\phi \vee \phi' \rightarrow \quad 1 - (1 - p_\phi)(1 - p_{\phi'})$
    - for all $x \in \{0,1\}^n$ we have $p_\phi(x) = \phi(x)$
  - degree $d \leq |\phi|$
  - can compute $p_\phi(x)$ in poly time from ϕ and x



The power of **IP**

- First step: **arithmetization**
  - transform $\phi(x_1, \ldots x_n)$ into polynomial $p_\phi(x_1, x_2, \ldots x_n)$ of degree d over a field GF(q); q prime > 2^n
  - recursively:
    - $x_i \rightarrow x_i$ $\qquad\qquad$ $\sim\phi \rightarrow (1 - p_\phi)$
    - $\phi \wedge \phi' \rightarrow (p_\phi)(p_{\phi'})$
    - $\phi \vee \phi' \rightarrow 1 - (1 - p_\phi)(1 - p_{\phi'})$
    - for all $x \in \{0,1\}^n$ we have $p_\phi(x) = \phi(x)$
  - degree $d \leq |\phi|$
  - can compute $p_\phi(x)$ in poly time from ϕ and x

Example: transfer $(x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_1 \vee \sim x_3)$ into polynomial



Idea:

- Recall the fundamental theorem of algebra:
  - every non-zero, single-variable, degree d polynomial *(with complex coefficients)* has, counted with multiplicity, exactly d roots

- So it's hard to cheat for polynomials, since if we randomly select $a \in GF(q)$, with prob. $\geq 1-d/q$, we can catch non-identical polynomials
  - See last lecture



#SAT = { (ϕ, k): CNF ϕ has exactly k satisfying assignments}

Another Try

- Prover wishes to prove:

$$k = \Sigma_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi(x_1, x_2, \ldots, x_n)$$

- Define: $k_z = \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi(z, x_2, \ldots, x_n)$
- prover sends: polynomial k_z
- verifier:
  - checks that $k_0 + k_1 = k$
  - sends random $a \in GF(q)$
- continue with proof that

$$k_a = \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi(a, x_2, \ldots, x_n)$$

- at end: verifier checks for itself



The power of **IP**

- Prover wishes to prove:

$$k = \Sigma_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi (x_1, x_2, \ldots, x_n)$$

- Define: $k_z = \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi (z, x_2, \ldots, x_n)$

Sumcheck protocol

If n=1, verifier checks whether p(1)+p(0)=k
Else
Prover sends a one-variable polynomial s(z)
which suppose to be k_z
Verifier rejects if $s(0)+s(1) \neq k$,
otherwise picks a random variable a in GF(q), recursively check whether
$s(a)=\Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} p(a, x_2, \ldots, x_n)$



Analysis of protocol

- Completeness:
  - if $(\phi, k) \in$ #SAT then honest prover on previous slide will always cause verifier to accept



Sumcheck protocol

If n=1, verifier checks whether p(1)+p(0)=K
Else
Prover sends a one-variable polynomial s(z)
which suppose to be k_z
Verifier rejects if s(0)+s(1)≠k,
otherwise picks a random variable a in GF(q), recursively check whether
$s(a)=\Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} p(a, x_2, \ldots, x_n)$

- Soundness:
  - when $k \neq \Sigma_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} p(x_1, x_2, \ldots, x_n)$,
  - if the prover honestly sends $s(z)=k_z$, then the verifier will reject immediately



Sumcheck protocol

If n=1, verifier checks whether p(1)+p(0)=K
Else
Prover sends a one-variable polynomial s(z)
which suppose to be k_z
Verifier rejects if $s(0)+s(1)\neq k$,
otherwise picks a random variable a in GF(q), recursively check whether
$s(a)=\Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} p(\mathbf{a}, x_2, \ldots, x_n)$

- Soundness:
  - when $k \neq \Sigma_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} p(x_1, x_2, \ldots, x_n)$,
  - if the prover cheats by sending an s(z) such that s(0)+s(1)=k,
  - then $s(z) \neq k_z = \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} p(\mathbf{z}, x_2, \ldots, x_n)$



Sumcheck protocol

If n=1, verifier checks whether p(1)+p(0)=K
Else
Prover sends a one-variable polynomial s(z)
which suppose to be k_z
Verifier rejects if $s(0)+s(1)\neq k$,
otherwise picks a random variable a in GF(q), recursively check whether
$s(a)=\Sigma_{x_2=0,1}\cdots\Sigma_{x_n=0,1}P(\mathbf{a}, x_2, \ldots, x_n)$

- Soundness:
  - when $k \neq \Sigma_{x_1=0,1}\Sigma_{x_2=0,1}\cdots\Sigma_{x_n=0,1}P(x_1, x_2, \ldots, x_n)$,
  - if the prover *cheats* by sending an s(z) such that s(0)+s(1)=k,
  - then s(z) is *not identical to* $k_z = \Sigma_{x_2=0,1}\cdots\Sigma_{x_n=0,1}P(\mathbf{z}, x_2, \ldots, x_n)$
  - so the number of a s.t. $s(a)= \Sigma_{x_2=0,1}\cdots\Sigma_{x_n=0,1}P(\mathbf{a}, x_2, \ldots, x_n)$ is the number of roots of $s(z)-k_z$, which is at most the degree d
  - So the probability that $s(a)\neq \Sigma_{x_2=0,1}\cdots\Sigma_{x_n=0,1}P(\mathbf{a}, x_2, \ldots, x_n)$ is $\geq(1-d/q)\geq(1-d/2^n)$



Analysis of protocol

- Soundness (continued):
  - recall that $d \leq |\phi|$
  - union bound: probability that the inequality always holds for n recursions is:

$$\geq (1-d/2^n)^n \geq 1-n|\phi|/2^n \geq 1-poly(n)/2^n >> 2/3$$

Usually $|\phi| << 2^n$, since otherwise we can check all assignments



Analysis of protocol

- Conclude: #SAT = { (ϕ, k): CNF ϕ has exactly k satisfying assignments} is in **IP**
- #SAT is **coNP**-hard, so **coNP**⊆**IP**
- Question remains:
  - **NP, coNP ⊆ IP.** Potentially larger. How much larger?



IP = PSPACE

Theorem: [Shamir 1990] **IP = PSPACE**

- Note: **IP** $\subseteq$ **PSPACE**
  - enumerate all possible interactions, explicitly calculate acceptance probability
- interaction extremely powerful!
- An implication: you can interact with master player of Reversi and determine if you can win from the current configuration even if you do not have the power to compute optimal moves!



IP = PSPACE

- need to prove **PSPACE ⊆ IP**
  - use same type of protocol as for **coNP**
  - some modifications needed



IP = PSPACE

- protocol for QSAT
  - arithmetization step produces **arithmetic expression** p_ϕ:
    - $(\exists\, x_i)\, \phi \longrightarrow \Sigma_{x_i = 0, 1}\, P_\phi$
    - $(\forall\, x_i)\, \phi \longrightarrow \Pi_{x_i = 0, 1}\, P_\phi$



IP = PSPACE

- protocol for QSAT
  - arithmetization step produces **arithmetic expression** p_ϕ:
    - $(\exists\, x_i)\, \phi \rightarrow \Sigma_{x_i = 0,\,1}\, P_\phi$
    - $(\forall\, x_i)\, \phi \rightarrow \Pi_{x_i = 0,\,1}\, P_\phi$
  - then
    - TQBF: $\forall x_1 \exists x_2 \forall x_3 \ldots\ \exists x_n\ \phi(x_1, x_2, \ldots, x_n)$
    - equivalent to:

$$\Pi_{x_1 = 0,\,1} \Sigma_{x_2 = 0,1}\, \Pi_{x_3 = 0,1} \cdots \Sigma_{x_n = 0,\,1} P_\phi(x_1, x_2, \ldots, x_n) > 0$$



IP = PSPACE

- TQBF: $\forall x_1 \exists x_2 \forall x_3 \ldots \exists x_n \, \phi(x_1, x_2, \ldots, x_n)$
- equivalent to:

$$\Pi_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \Pi_{x_3 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi(x_1, x_2, \ldots, x_n) > 0$$

- can we use the previous proof for #SAT to prove this?
- Why?



IP = PSPACE

- TQBF: $\forall x_1 \exists x_2 \forall x_3 \ldots \exists x_n \, \phi(x_1, x_2, \ldots, x_n)$
- equivalent to:

$$\Pi_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \Pi_{x_3 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi(x_1, x_2, \ldots, x_n) > 0$$

- can we use the previous proof for #SAT to prove this?
- No, since the prover needs to send the polynomial s(z), but here the degree can be $2^{\Omega(n)}$

Sumcheck protocol

If n=1, verifier checks whether p(1)+p(0)=K
Else
Prover sends a one-variable polynomial s(z)
which suppose to be k_z
Verifier rejects if $s(0)+s(1) \neq k$,
otherwise picks a random variable a in GF(q), recursively check whether
$s(a) = \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} P(\mathbf{a}, x_2, \ldots, x_n)$



IP = PSPACE

- TQBF: $\forall x_1 \exists x_2 \forall x_3 \ldots \exists x_n \, \phi(x_1, x_2, \ldots, x_n)$
- equivalent to:

$$\Pi_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \Pi_{x_3 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi(x_1, x_2, \ldots, x_n) > 0$$

- can we use the previous proof for #SAT to prove this?
- No, since the prover needs to send the polynomial s(z), but here the degree can be $2^{\Omega(n)}$
  - Originally all sums $\Sigma_{x_1 = 0,1} \Sigma_{x_2 = 0,1} \cdots \Sigma_{x_n = 0,1} P_\phi$, so degree does not increase from P_ϕ
  - Here a product can make the degree double

- However, remember we only care about $z \in \{0,1\}$
- so z^k can be seen as just z



Degree Reduction Operator

- $Rx_iU(x_1,\ldots x_i,\ldots,x_n) = U$ with all $x_i^k \rightarrow x_i$

- Let old ϕ (without R)= ϕ_0
  - $(\exists\, x_i)\, \phi \rightarrow \Sigma_{x_i = 0,\,1}\, P_\phi$
  - $(\forall\, x_i)\, \phi \rightarrow \Pi_{x_i = 0,\,1}\, P_\phi$
  - $(Rx_i)\, \phi(x_1,\ldots x_i,\ldots,x_n) \rightarrow x_i \cdot p_\phi(x_1\ldots x_{i-1},1,x_{i+1},\ldots x_n) +$
    $(1-x_i) \cdot p_\phi(x_1\ldots x_{i-1},0,x_{i+1},\ldots x_n)$

- Let

$\phi = \forall x_1 Rx_1\ \exists x_2 Rx_1 Rx_2 \forall x_3 Rx_1 Rx_2 Rx_3 \ldots\ \exists x_n Rx_1 Rx_2 \ldots Rx_n\ \phi_0(x_1,\ldots,x_n)$

Thus, p_ϕ keeps a multilinear function



Degree Reduction Operator

- $Rx_iU(x_1,\ldots x_i,\ldots,x_n) = U$ with all $x_i^k \rightarrow x_i$

$(Rx)(A+Bx+Cx^2+Dx^3)$
$= x(A+B+C+D)+(1-x)\cdot A$
$= A+(B+C+D)x$

- Let old ϕ (without R)= ϕ_0
  - $(\exists\, x_i)\ \phi \rightarrow \Sigma_{x_i = 0,\,1}\ P_\phi$
  - $(\forall\, x_i)\ \phi \rightarrow \prod_{x_i = 0,\,1}\ P_\phi$
  - $(Rx_i)\ \phi(x_1,\ldots x_i,\ldots,x_n) \rightarrow x_i \cdot p_\phi(x_1\ldots x_{i-1},1,x_{i+1},\ldots x_n)+$
    $(1-x_i)\cdot p_\phi(x_1\ldots x_{i-1},0,x_{i+1},\ldots x_n)$
- Let

$\phi = \forall x_1 Rx_1\ \exists x_2 Rx_1 Rx_2 \forall x_3 Rx_1 Rx_2 Rx_3 \ldots\ \exists x_n Rx_1 Rx_2 \ldots Rx_n\ \phi_0(x_1,\ldots,x_n)$

Thus, p_ϕ keeps a multilinear function



The power of **IP**

- Prover wishes to prove:

$$k = p_{\phi}$$

- Define: $\phi=(Ox_1)\ \phi'(x_1)$, where O= ∀, ∃, or R
  - when O=R, x_1 is already assigned a_1

TQBF protocol

Prover sends a one-variable polynomial $s(x_1)$

which suppose to be $p_{\phi'}(x_1)$

Verifier checks:

if O= ∀, check $s(0)\cdot s(1)=k$

if O= ∃, check $s(0)+s(1)=k$

if O= R, check $a_1 s(1)+(1-a_1)s(0)=k$

if it is not equal, rejects; otherwise picks a random variable a in GF(q), recursively check whether $s(a)= p_{\phi'}(a)$



The power of **IP**

EXAMPLE: $\forall x_1 R x_1 \exists x_2 R x_1 R x_2 \forall x_3 R x_1 R x_2 R x_3 \, (x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_1 \vee \sim x_3)$

- Prover wishes to prove:

$$k = p_\phi$$

- Define: $\phi = (O x_1)\, \phi'(x_1)$, where $O = \forall, \exists$, or R
  - when $O = R$, x_1 is already assigned a_1

TQBF protocol

Prover sends a one-variable polynomial $s(x_1)$

which suppose to be $p_{\phi'}(x_1)$

Verifier checks:

if $O = \forall$, check $s(0) \cdot s(1) = k$

if $O = \exists$, check $s(0) + s(1) = k$

if $O = R$, check $a_1 s(1) + (1 - a_1) s(0) = k$

if it is not equal, rejects, otherwise picks a random variable a in GF(q), recursively check whether $s(a) = p_{\phi'}(a)$

$\phi = \forall x_1 R x_1 \; \exists x_2 R x_1 R x_2 \forall x_3 R x_1 R x_2 R x_3 \ldots \; \exists x_n R x_1 R x_2 \ldots R x_n$

When we reach (Rx_i), x_i has already been assigned a value by

Example

- $\forall x_1 Rx_1 \exists x_2 Rx_1 Rx_2 \forall x_3 Rx_1 Rx_2 Rx_3 (x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_1 \vee \sim x_3)$
  - $p_{\phi 0} = 1 - x_2 + x_1 x_2 + x_2 x_3 - x_1 x_3 - x_1^2 x_2 x_3 - x_1 x_2 x_3^2 + x_1^2 x_2 x_3^2$
  - $Rx_3(\phi_0)$: $1 - x_2 + x_1 x_2 + x_2 x_3 - x_1 x_3 - x_1 x_2 x_3$
  - $\forall x_3 Rx_1 Rx_2 Rx_3\ (\phi_0)$: $1 - x_1 - x_2 + 2x_1 x_2 - x_1^2 x_2$
- Before it reaches some $\forall$ or $\exists$, the polynomial will be multilinear, so degree is linear to $|\phi|$



- $\forall x_1 R x_1 \exists x_2 R x_1 R x_2 \forall x_3 R x_1 R x_2 R x_3 (x_1 \vee \sim x_2 \vee x_3) \wedge (\sim x_1 \vee \sim x_3)$
  - $p_{\phi 0} = 1 - x_2 + x_1 x_2 + x_2 x_3 - x_1 x_3 - x_1^2 x_2 x_3 - x_1 x_2 x_3^2 + x_1^2 x_2 x_3^2$
  - $Rx_3(\phi_0)$: $1 - x_2 + x_1 x_2 + x_2 x_3 - x_1 x_3 - x_1 x_2 x_3$

  - verify whether $k = 1 - a_2 + a_1 a_2 + a_2 a_3 - a_1 a_3 - a_1 a_2 a_3$
  - $s(x_3)$ suppose to be $1 - a_2 + a_1 a_2 + a_2 x_3 - a_1 x_3 - a_1^2 a_2 x_3 - a_1 a_2 x_3^2 + a_1^2 a_2 x_3^2$
  - check $a_3 s(1) + (1 - a_3) s(0) = k$
  - after that we select another a_3' to verify whether

    $s(a_3') = 1 - a_2 + a_1 a_2 + a_2 a_3' - a_1 a_3' - a_1^2 a_2 a_3' - a_1 a_2 a_3'^2 + a_1^2 a_2 a_3'^2$

Degree of s is bounded by $O(|\phi|)$

TQBF protocol

Prover sends a one-variable polynomial $s(x_1)$

which suppose to be $p_{\phi'}(x_1)$

Verifier checks:

if $O = \forall$, check $s(0) \cdot s(1) = k$

if $O = \exists$, check $s(0) + s(1) = k$

if $O = R$, check $a_1 s(1) + (1 - a_1) s(0) = k$

if it is not equal, rejects, otherwise picks a random variable a in GF(q), recursively check whether $s(a) = p_{\phi'}(a)$



Analysis of protocol

- Completeness:
  - if $\phi \in$ TQBF then honest prover on previous slide will always cause verifier to accept



- Soundness:
  - Similar to the protocol of #SAT, the probability that $s(a) \neq p_{\phi'}(a)$ is at least $(1-d/q) \geq (1-d/2^n)$
  - probability that the inequality always holds for $<n^2$ recursions is:

$$\geq \left(1 - \frac{d}{2^n}\right)^{n^2} \geq 1\text{-poly}(n)/2^n >> 2/3$$



PSPACE⊆IP

- **interactive proof system** for L is an interactive protocol (P,V)
  - completeness: $x \in L \Rightarrow \exists P$

    $Pr_r[V \text{ accepts in } (P,V)(x)] \geq 2/3$
  - soundness: $x \notin L \Rightarrow \forall P^*$

    $Pr_r[V \text{ accepts in } (P^*,V)(x)] \leq 1/3$
  - efficiency: V is p.p.t. machine based on random variable r
  - P cannot see r
- repetition: can reduce error to any ε

- *"Private coins model"*



Interactive Proofs

- **interactive proof system** for L is an interactive protocol (P,V)
  - completeness: $x \in L \Rightarrow \exists P$

    $Pr_r[V \text{ accepts in } (P,V)(x)] = 1$
  - soundness: $x \notin L \Rightarrow \forall P^*$

    $Pr_r[V \text{ accepts in } (P^*,V)(x)] \leq 1/3$
  - efficiency: V is p.p.t. machine based on random variable r
  - P cannot see r
- repetition: can reduce error to any ε

- *"Private coins model"*



Interactive Proofs

- **interactive proof system** for L is an interactive protocol (P,V)
  - completeness: $x \in L \Rightarrow \exists P$

    $Pr_r[V \text{ accepts in } (P,V)(x)] \geq 2/3$
  - soundness: $x \notin L \Rightarrow \forall P^*$

    $Pr_r[V \text{ accepts in } (P^*,V)(x)] = 0$
  - efficiency: V is p.p.t. machine based on random variable r
  - P cannot see r
- repetition: can reduce error to any ε

- *"Private coins model"*



Arthur-Merlin Games

- **IP** permits verifier to keep coin-flips private
  - necessary feature?
  - GNI protocol breaks without it

- Arthur-Merlin game: interactive protocol in which coin-flips are public
  - Arthur (verifier) may as well just send results of coin-flips and ask Merlin (prover) to perform any computation Arthur would have done



Arthur-Merlin Games

- Clearly **Arthur-Merlin ⊆ IP**
  - In public coins model, prover can see all the random bits
  - In private coins model, verifier may choose what to send to the prover, so it can also send all random bits

  - "private coins are at least as powerful as public coins"



Arthur-Merlin Games

- Clearly **Arthur-Merlin ⊆ IP**
  - "private coins are at least as powerful as public coins"

- Proof that **IP** = **PSPACE** actually shows

PSPACE ⊆ Arthur-Merlin

TQBF protocol

Prover sends a one-variable polynomial $s(x_1)$

which suppose to be $p_{\phi'}(x_1)$

Verifier checks:

if O= ∀, check $s(0) \cdot s(1)=k$

if O= ∃, check $s(0)+s(1)=k$

if O= R, check $a_1 s(1)+(1-a_1)s(0)=K$

if it is not equal, reject, otherwise picks a random variable a in GF(q), recursively check whether $s(a)= p_{\phi'}(a)$

We can just send a to Prover

public coins only!

Arthur-Merlin Games

- Clearly **Arthur-Merlin ⊆ IP**
  - "private coins are at least as powerful as public coins"

- Proof that **IP** = **PSPACE** actually shows

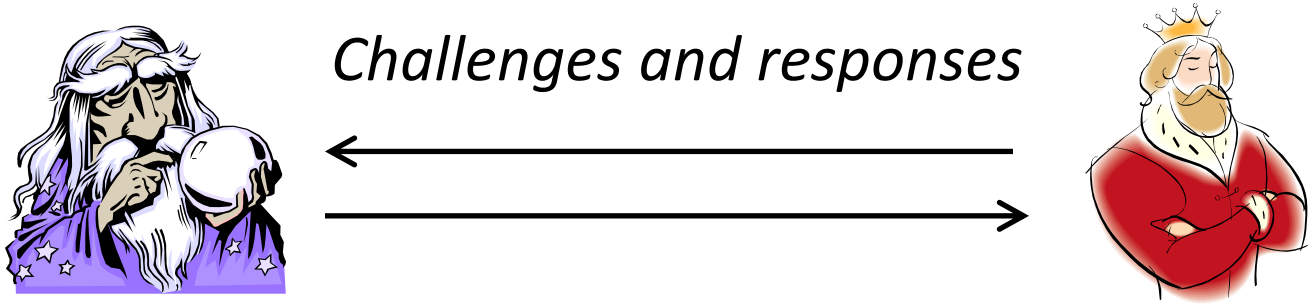
  PSPACE ⊆ Arthur-Merlin ⊆ IP ⊆ PSPACE

  - "public coins are at least as powerful as private coins!"



Arthur-Merlin Games

- Delimiting # of rounds:
  - **AM[k]** = Arthur-Merlin game with k messages, Arthur (verifier) goes first
  - **MA[k]** = Arthur-Merlin game with k messages, Merlin (prover) goes first



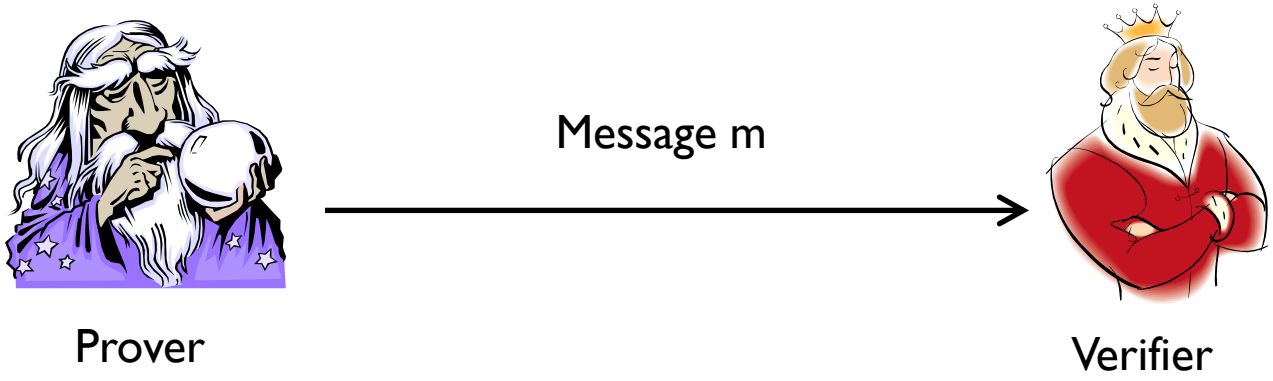
Merlin: *Omniscient but untrustworthy wizard*

Arthur: *Skeptical, polynomial-time king*



- Delimiting # of rounds:
  - **AM[k]** = Arthur-Merlin game with k messages, Arthur (verifier) goes first
  - **MA[k]** = Arthur-Merlin game with k messages, Merlin (prover) goes first

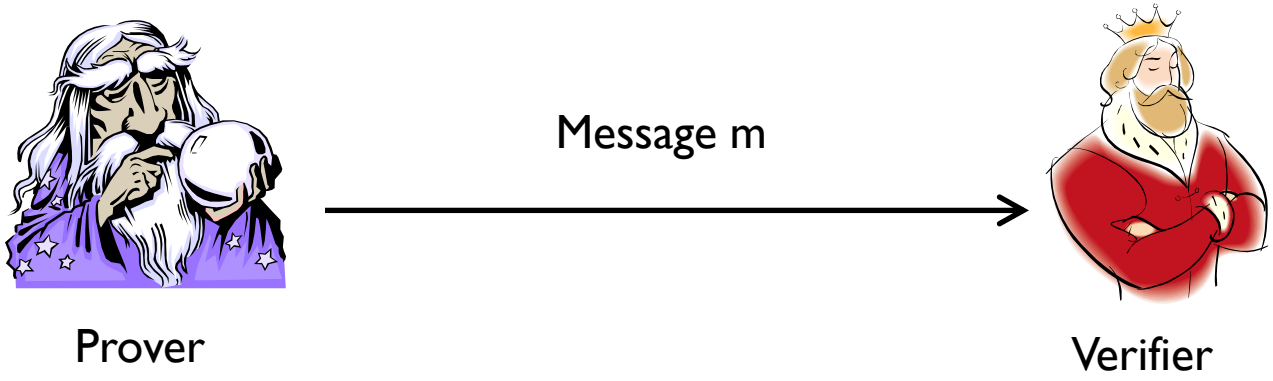
MA[2]: 1-message protocol where Merlin sends Arthur a message, and then Arthur decides whether to accept or not by running a probabilistic polynomial time computation. (similar to NP, just the verifier can use randomness.)





- Delimiting # of rounds:
  - **AM[k]** = Arthur-Merlin game with k messages, Arthur (verifier) goes first
  - **MA[k]** = Arthur-Merlin game with k messages, Merlin (prover) goes first

MA[2]: 1-message protocol where Merlin sends Arthur a message, and then Arthur decides whether to accept or not by running a probabilistic polynomial time computation. (similar to NP, just the verifier can use randomness.)



$L \in \mathbf{MA[2]}$ iff $\exists$ poly-time language R

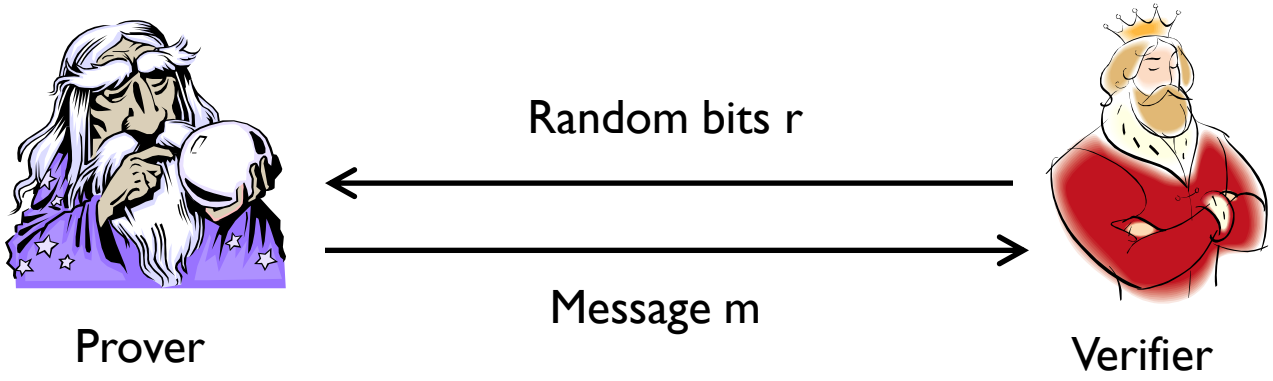
$x \in L \Rightarrow \exists\, m\ Pr_r[(x, m, r) \in R] = 1$

$x \notin L \Rightarrow \forall\, m\ Pr_r[(x, m, r) \in R] \leq \frac{1}{2}$



- Delimiting # of rounds:
  - **AM[k]** = Arthur-Merlin game with k messages, Arthur (verifier) goes first
  - **MA[k]** = Arthur-Merlin game with k messages, Merlin (prover) goes first

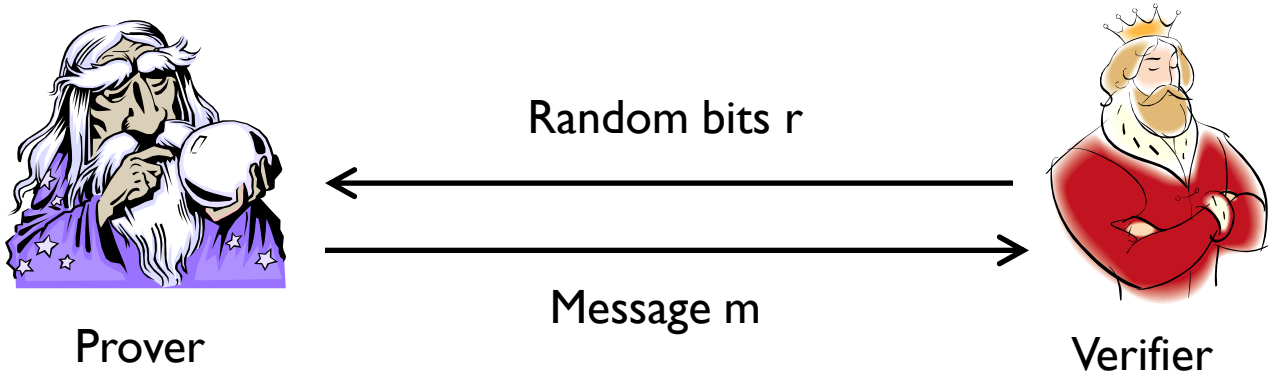
AM[2]: Arthur tosses some random coins and sends the outcome of *all* his coin tosses to Merlin, Merlin responds with a purported proof, and Arthur deterministically verifies the proof.





- Delimiting # of rounds:
  - **AM[k]** = Arthur-Merlin game with k messages, Arthur (verifier) goes first
  - **MA[k]** = Arthur-Merlin game with k messages, Merlin (prover) goes first

AM[2]: Arthur tosses some random coins and sends the outcome of *all* his coin tosses to Merlin, Merlin responds with a purported proof, and Arthur deterministically verifies the proof.



$L \in$ **AM[2]** iff $\exists$ poly-time language R

$x \in L \Rightarrow Pr_r[\exists\ m\ (x, m, r) \in R] = 1$

$x \notin L \Rightarrow Pr_r[\exists\ m\ (x, m, r) \in R] \leq \frac{1}{2}$



MA and **AM**

- Two important classes:
  - **MA = MA[2]**
  - **AM = AM[2]**
- definitions without reference to interaction:
  - L ∈ **MA** iff ∃ poly-time language R

    $x \in L \Rightarrow \exists\, m\ Pr_r[(x, m, r) \in R] = 1$

    $x \notin L \Rightarrow \forall\, m\ Pr_r[(x, m, r) \in R] \leq \frac{1}{2}$
  - L ∈ **AM** iff ∃ poly-time language R

    $x \in L \Rightarrow Pr_r[\exists\, m\ (x, m, r) \in R] = 1$

    $x \notin L \Rightarrow Pr_r[\exists\, m\ (x, m, r) \in R] \leq \frac{1}{2}$



Arthur-Merlin Games

- given L ∈**MA[2]**

$$x \in L \Rightarrow \exists m \; Pr_r[(x, m, r) \in R] = 1$$

$$\Rightarrow Pr_r[\exists m \; (x, m, r) \in R] \quad ?$$

$$x \notin L \Rightarrow \forall m \; Pr_r[(x, m, r) \in R] \leq \varepsilon$$

$$\Rightarrow Pr_r[\exists m \; (x, m, r) \in R] \quad ?$$



Arthur-Merlin Games

- given $L \in$ **MA[2]**

$$x \in L \Rightarrow \exists m \; Pr_r[(x, m, r) \in R] = 1$$

$$\Rightarrow Pr_r[\exists m \; (x, m, r) \in R] = 1$$

$$x \notin L \Rightarrow \forall m \; Pr_r[(x, m, r) \in R] \leq \varepsilon$$

$$\Rightarrow Pr_r[\exists m \; (x, m, r) \in R] \leq 2^{|m|}\varepsilon$$

- by sending t times the independent random strings r, we can make error exponentially small
- set $t = m+1$ to get $2^{|m|}\varepsilon < ½$

- Thus **MA[2]** $\subseteq$ **AM[2]**



Arthur-Merlin Games

- given L ∈**MA[2]**

$$x \in L \Rightarrow \exists m \Pr_r[(x, m, r) \in R] = 1$$

$$\Rightarrow \Pr_r[\exists m\ (x, m, r) \in R] = 1$$

$$x \notin L \Rightarrow \forall m \Pr_{r_1,\ldots,r_t}[\mathrm{V}_{i=1}^{t}(x, m, r_i) \in R] \leq \varepsilon^t$$

$$\Rightarrow \Pr_{r_1,\ldots,r_t}[\exists m\, \mathrm{V}_{i=1}^{t}(x, m, r_i) \in R] \leq 2^{|m|}\varepsilon^t$$

- by sending t times the independent random strings r, we can make error exponentially small
- set $t = m+1$ to get $2^{|m|}\varepsilon < ½$

- Thus **MA[2]** ⊆ **AM[2]**



Arthur-Merlin Games

Theorem: for all constant $k \geq 2$

$$\mathbf{AM[k] = AM[2]}$$

- Proof:
  - we first show **MA[2] ⊆ AM[2]**
  - implies can move all of Arthur's messages to beginning of interaction:

$$AMAMAM\ldots AM \subseteq AAMMAM\ldots AM \subseteq \ldots \subseteq AAA\ldots AMMM\ldots M$$

Every switch of MA to AM will increase error prob. by $2^{|m|}$ times,
which will multiply the iteration by O(m), so this can repeat for constant times



MA and **AM**

- $L \in$ **AM** iff $\exists$ poly-time language R

$$x \in L \Rightarrow Pr_r[\exists\, m\ (x, m, r) \in R] = 1$$

$$x \notin L \Rightarrow Pr_r[\exists\, m\ (x, m, r) \in R] \leq \tfrac{1}{2}$$

- Relation to other complexity classes:
  - both contain **NP** (can elect to not use randomness)
  - both contained in $\prod_2$. $L \in \prod_2$ iff $\exists R \in$ **P** for which:

$$x \in L \Rightarrow Pr_r[\exists m\ (x, m, r) \in R] = 1$$

$$x \notin L \Rightarrow Pr_r[\exists m\ (x, m, r) \in R] < 1$$

  - so clearly that **AM** $\subseteq \prod_2$
  - known that **MA** $\subseteq$ **AM**



Perfect completeness

- L ∈ **MA** iff ∃ poly-time language R

$$x \in L \Rightarrow \exists\, m\ Pr_r[(x, m, r) \in R] = 1$$

$$x \notin L \Rightarrow \forall\, m\ Pr_r[(x, m, r) \in R] \leq \tfrac{1}{2}$$

- What about:

$$x \in L \Rightarrow \exists\, m\ Pr_r[(x, m, r) \in R] \geq 2/3$$
$$x \notin L \Rightarrow \forall\, m\ Pr_r[(x, m, r) \in R] \leq 1/3$$

equals to MA, proof similar to BPP ⊆ Π_2 in the last lecture

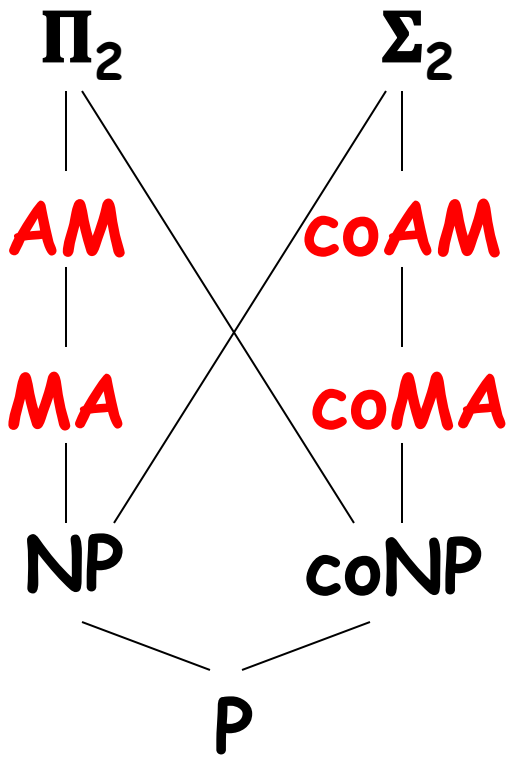
$$x \in L \Rightarrow \exists\, m\ Pr_r[(x, m, r) \in R] \geq 2/3$$
$$x \notin L \Rightarrow \forall\, m\ Pr_r[(x, m, r) \in R] = 0$$

equals to NP

- Similar for AM



MA and **AM**



Proof sketch of GNI∈AM

- No private coins, so the proof of GNI∈IP doesn't work!
- Convince the verifier that G_0 is not isomorphic to G_1

- Define $S=\{(H, \pi): H \cong G_0 \text{ or } H \cong G_1 \text{ and } \pi \in aut(H)\}$
  - where aut(H) is the set of permutation π such that $\pi(H)=H$



Proof sketch of GNI∈AM

- No private coins, so the proof of GNI∈IP doesn't work!
- Convince the verifier that G_0 is not isomorphic to G_1

- Define $S=\{(H, \pi): H \cong G_0$ or $H \cong G_1$ and $\pi \in aut(H)\}$
  - where aut(H) is the set of permutation π such that π(H)=H

- So if $G_0 \cong G_1$, $|S|=n!$, otherwise $|S|=2n!$
  - So we need "set lower bound protocol"



Set lower bound protocol

- Certify the approximate size of S
  - we need to check whether $|S| \geq 2^k$ or $|S| \leq 2^{k-1}$
  - $S \subseteq \{0,1\}^m$

- membership of S in NP



Pairwise independent hash functions

- a family H of functions $\{0,1\}^m \rightarrow \{0,1\}^k$
  - given x,y, $Pr_{h \in H}[h(x)=y]=2^{-k}$

- pairwise independent:
  - for every $x,x' \in \{0,1\}^m$ with $x \neq x'$,
  - for every $y,y' \in \{0,1\}^k$
  - $Pr_{h \in H}[h(x)=y \text{ and } h(x')=y']=2^{-2k}$



Pairwise independent hash functions

- a family H of functions $\{0,1\}^m \rightarrow \{0,1\}^k$
  - given x,y, $Pr_{h \in H}[h(x)=y]=2^{-k}$

- pairwise independent:
  - for every $x,x' \in \{0,1\}^m$ with $x \neq x'$,
  - for every $y,y' \in \{0,1\}^k$
  - $Pr_{h \in H}[h(x)=y \text{ and } h(x')=y']=2^{-2k}$

In finite field $GF(2^m)$, an example of H(x) is ax+b for $a,b \in GF(2^m)$, and we can truncate the last m-k bits to make it k bits

from $ax+b=y$, $ax'+b=y'$, we get $a=(y-y')(x-x')^{-1}$ and $b=y-ax$



- If we need to check $|S| \geq 2^k$ or $|S| \leq 2^{k-1}$
  - $S \subseteq \{0,1\}^m$

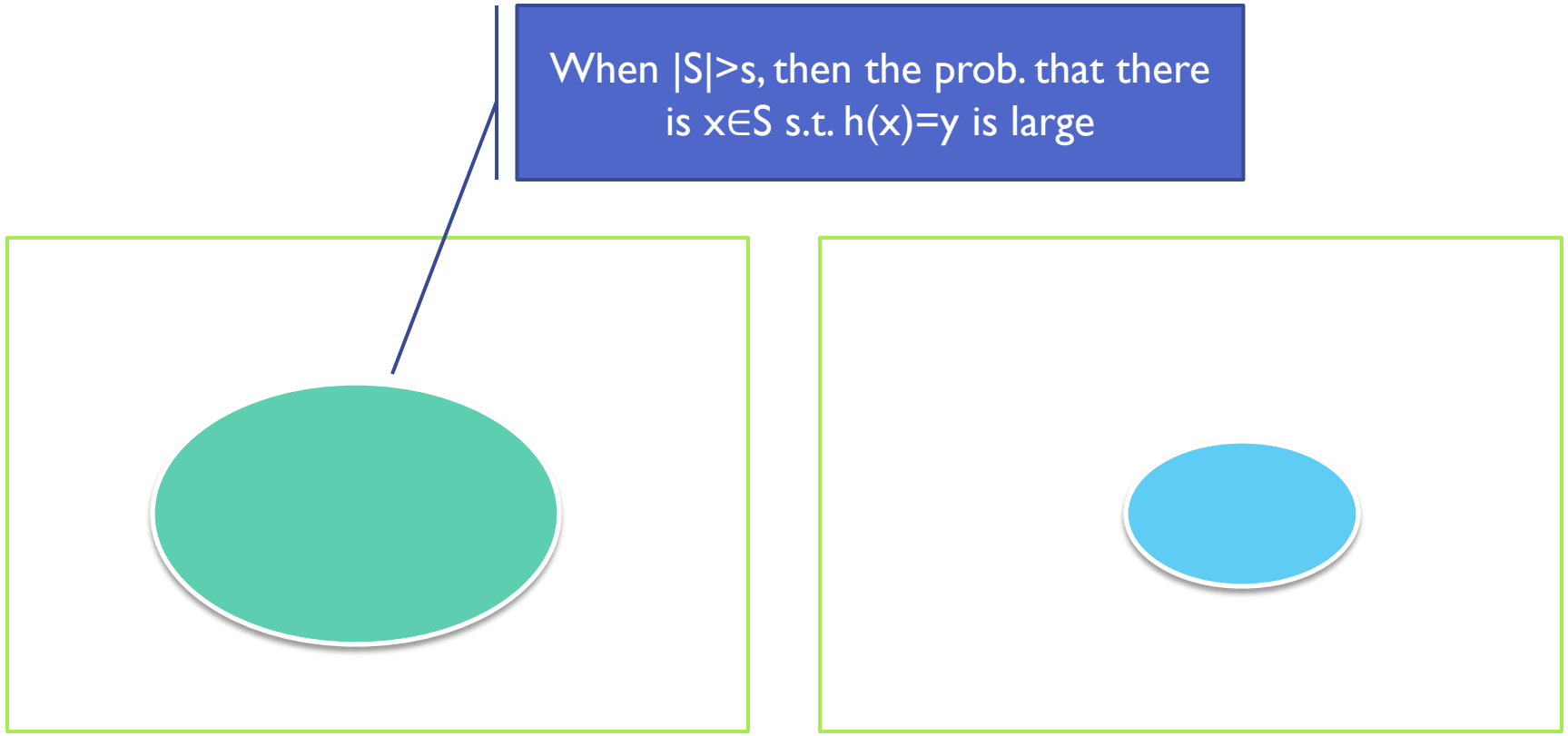
- Goldwasser-Sipser set lower bound protocol:

Verifier: randomly pick a *pairwise independent* hash function $h:\{0,1\}^m \to \{0,1\}^{k+1}$, and randomly pick $y \in \{0,1\}^{k+1}$
Prover: send $x \in S$ such that $h(x)=y$, and a certificate that $x \in S$
Verifier: Accept iff $h(x)=y$ and $x \in S$

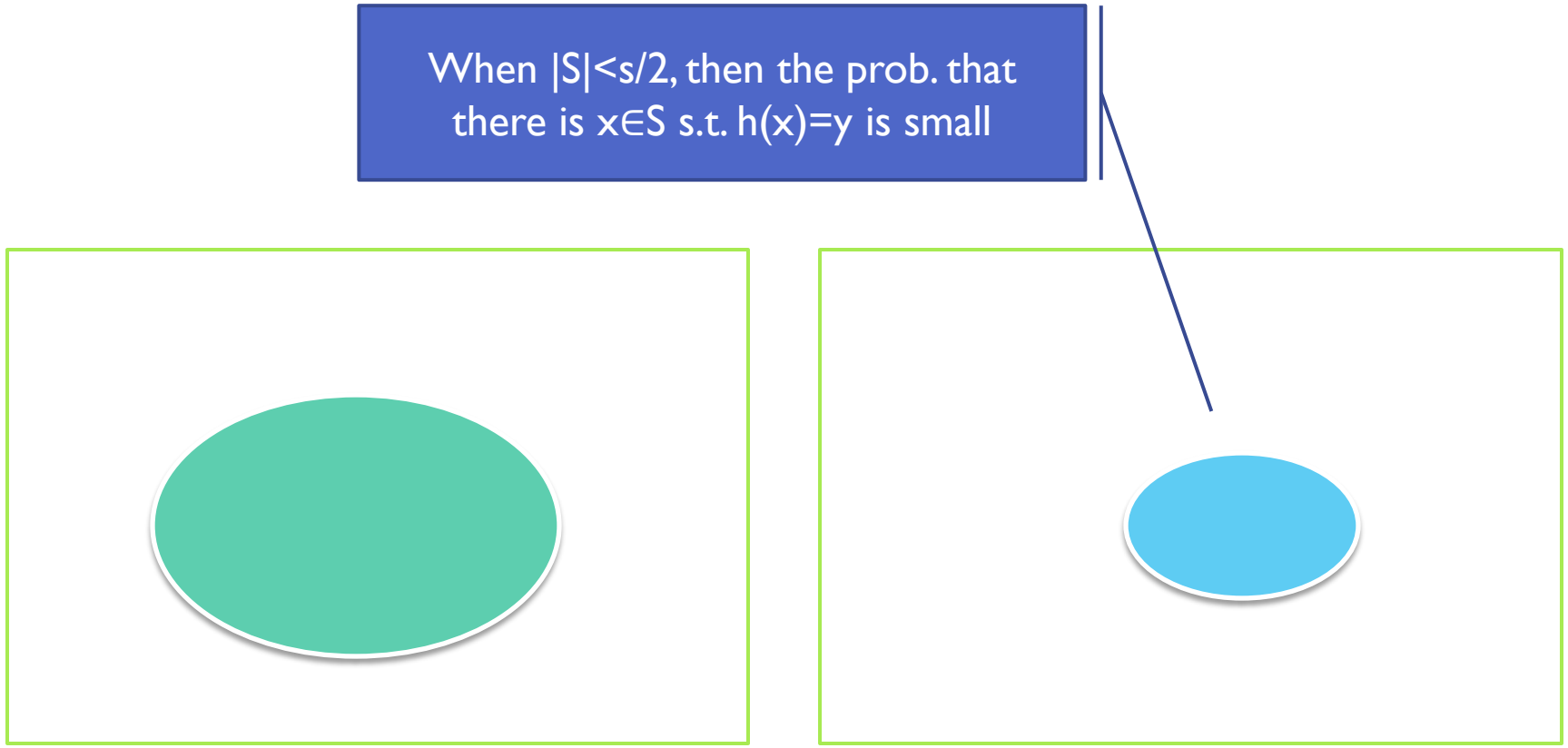
So if S is larger, the prob. that such an x exists is higher



- given any y, $Pr_h[h(x)=y]=1/s$



- given any y, $Pr_h[h(x)=y]=1/s$



- Verifier: randomly pick a *pairwise independent* hash function $h:\{0,1\}^m \to \{0,1\}^{k+1}$, and randomly pick $y \in \{0,1\}^{k+1}$
- Prover: send $x \in S$ such that $h(x)=y$, and a certificate that $x \in S$
- Verifier: Accept iff $h(x)=y$ and $x \in S$

We consider $\Pr_{h,y}[\exists x \in S: h(x)=y]$:

For all y,

$$\Pr_h[\exists x \in S: h(x)=y]$$



- Verifier: randomly pick a *pairwise independent* hash function $h:\{0,1\}^m \to \{0,1\}^{k+1}$, and randomly pick $y \in \{0,1\}^{k+1}$
- Prover: send $x \in S$ such that $h(x)=y$, and a certificate that $x \in S$
- Verifier: Accept iff $h(x)=y$ and $x \in S$

We consider $\Pr_{h,y}[\exists x \in S: h(x)=y]$:

For all y,

$$\Pr_h[\exists x \in S: h(x)=y] \le \sum_{x \in S} \Pr_h[h(x)=y] = |S|/2^{k+1}$$

$$\begin{aligned} &\Pr_h[\exists x \in S: h(x)=y] \\ &\ge \sum_{x \in S} \Pr_h[h(x)=y] - \tfrac{1}{2}\sum_{x \neq x' \in S} \Pr_h[h(x)=h(x')=y] \\ &\ge |S|/2^{k+1} - \tfrac{1}{2}|S|^2/2^{2k+2} \end{aligned}$$



- Verifier: randomly pick a *pairwise independent* hash function $h:\{0,1\}^m \to \{0,1\}^{k+1}$, and randomly pick $y \in \{0,1\}^{k+1}$
- Prover: send $x \in S$ such that $h(x)=y$, and a certificate that $x \in S$
- Verifier: Accept iff $h(x)=y$ and $x \in S$

We consider $\Pr_{h,y}[\exists x \in S: h(x)=y] = \Pr[V \text{ accepts}]$:

For all y,

$$\Pr_h[\exists x \in S: h(x)=y] \le \sum_{x \in S} \Pr_h[h(x)=y] = |S|/2^{k+1}$$

$$\begin{aligned} &\Pr_h[\exists x \in S: h(x)=y] \\ &\ge \sum_{x \in S} \Pr_h[h(x)=y] - \tfrac{1}{2}\sum_{x \neq x' \in S} \Pr_h[h(x)=h(x')=y] \\ &\ge |S|/2^{k+1} - \tfrac{1}{2}|S|^2/2^{2k+2} \end{aligned}$$

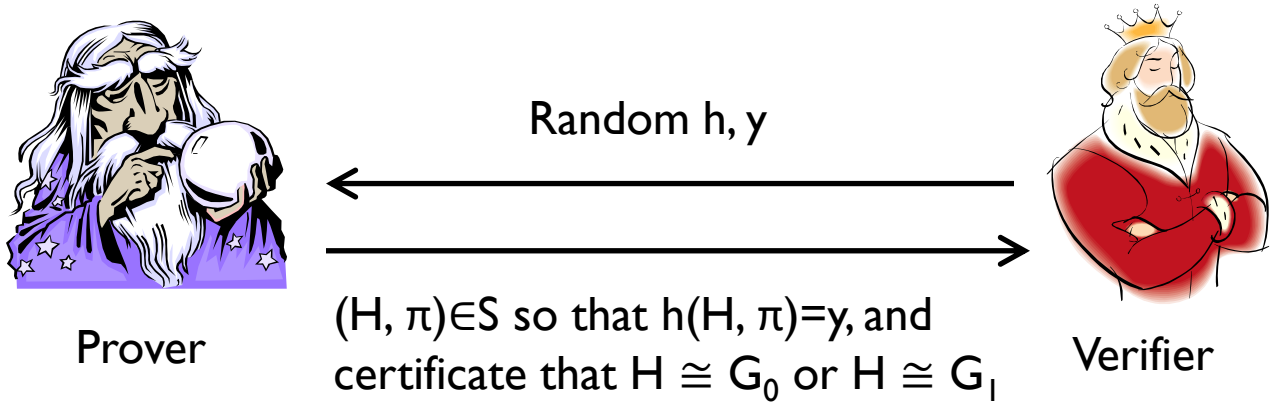
So when $|S|=2^k$, $\Pr[V \text{ accepts}] \ge 3/8$

when $|S|=2^{k-1}$, $\Pr[V \text{ accepts}] \le 1/4$



Proof sketch of GNI∈AM

- Define $S=\{(H, \pi): H \cong G_0$ or $H \cong G_1$ and $\pi \in aut(H)\}$
  - where aut(H) is the set of permutation π such that π(H)=H
- So if $G_0 \cong G_1$, |S|=n!, otherwise |S|=2n!
  - So we need "set lower bound protocol"





MA and **AM**

Theorem: **coNP ⊆ AM ⇒ PH = AM**

- Proof:
  - suffices to show $\Sigma_2 \subseteq$ **AM** (and use **AM** $\subseteq \Pi_2$)
  - L ∈ Σ_2 iff ∃ poly-time language R

    $x \in L \Rightarrow \exists\, y\ \forall\, z\ (x, y, z) \in R$

    $x \notin L \Rightarrow \forall\, y\ \exists\, z\ (x, y, z) \notin R$
  - Merlin sends y
  - 1 AM exchange decides **coNP** query: $\forall z\ (x, y, z) \in R$?



Back to Graph Isomorphism

- The payoff:
  - not known if GI is **NP**-complete
  - Theorem:

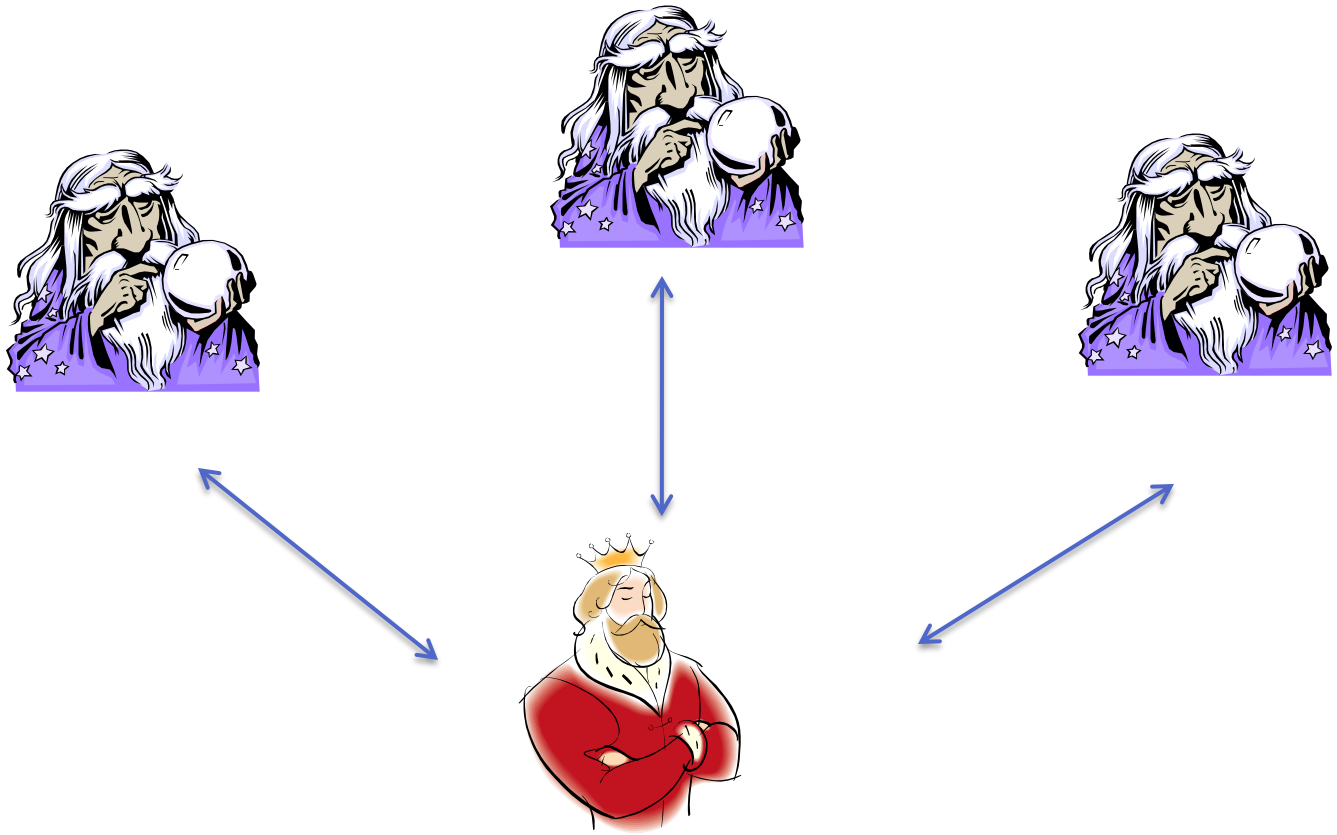
    if GI is **NP**-complete then **PH = AM**
  - so it is unlikely that GI is NP-complete!

  - Proof: GI **NP**-complete ⇒ GNI **coNP**-complete ⇒ **coNP⊆AM** ⇒ **PH = AM**
  - which means PH collapses to $\sum_2$



Multiprover Interactive Proofs

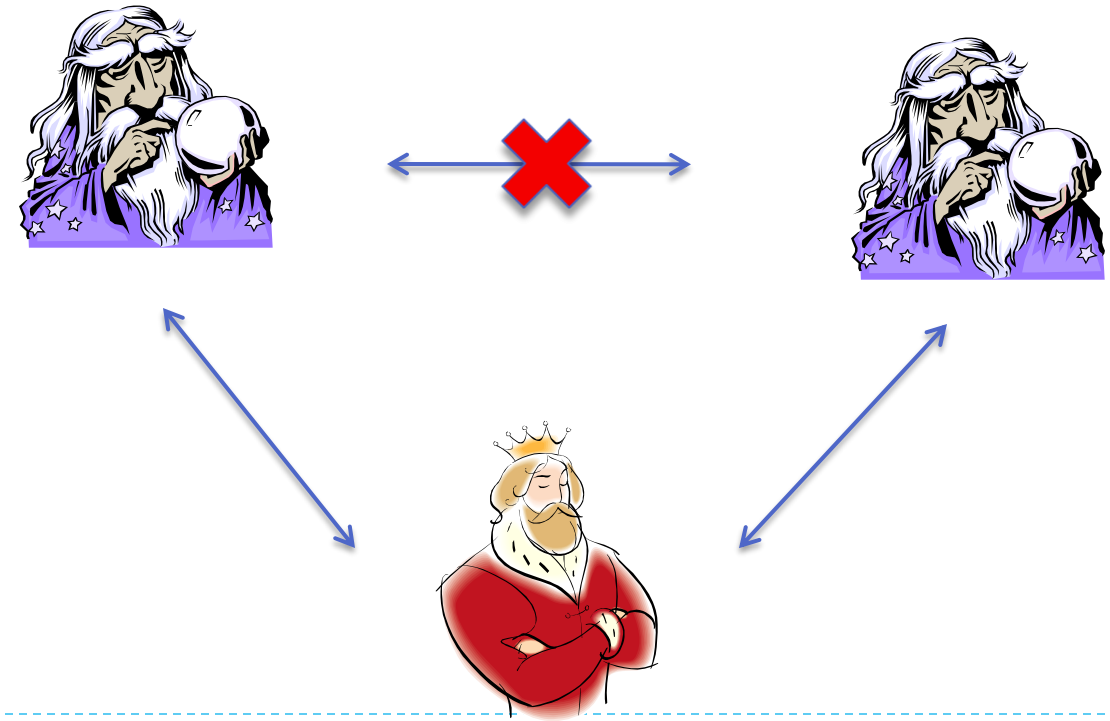
- What about there are two or more provers?





Multiprover Interactive Proofs

- Important: provers are not allowed to communicate with each other!
  - otherwise it's the same as only one prover!





Multiprover Interactive Proofs

- Important: provers are not allowed to communicate with each other!

- Thus, many books say it's similar to police interrogating suspects
  - Interrogating separately can get more information
  - So probably MIP is more powerful than IP



Power of MIP

- *[Babai, Fortnow, Lund 1991]* MIP=NEXP

- So MIP is strictly larger than NP



Probabilistic Oracle Machine

- Let M^O be a probabilistic polynomial time TM with access to an oracle O

- We say L is accepted by M^O iff:

$$x \in L \Rightarrow \exists\, O \Pr[M^O \text{ accepts } x] \geq 2/3$$

$$x \notin L \Rightarrow \forall\, O \Pr[M^O \text{ accepts } x] \leq 1/3$$

- What's the difference from IP?
  - The oracle can be seen as the prover?



Probabilistic Oracle Machine

- Let M^O be a probabilistic polynomial time TM with access to an oracle O.

- We say L is accepted by M^O iff:

$$x \in L \Rightarrow \exists\, O \Pr[M^O \text{ accepts } x] \geq 2/3$$

$$x \notin L \Rightarrow \forall\, O \Pr[M^O \text{ accepts } x] \leq 1/3$$

In IP, the prover can be adaptive. (It can make future answers depend on previous questions)

But the oracle must be set ahead of time

Big difference?



Probabilistic Oracle Machine

- Let M^O be a probabilistic polynomial time TM with access to an oracle O.

- We say L is accepted by M^O iff:

$$x \in L \Rightarrow \exists\, O \Pr[M^O \text{ accepts } x] \geq 2/3$$

$$x \notin L \Rightarrow \forall\, O \Pr[M^O \text{ accepts } x] \leq 1/3$$

- It's easy to see: (polynomial-prover) MIP is a subset of the language of Prob. Oracle Machine:
  - Oracle O: can answer query $(i, j, \beta_{i1}, \ldots \beta_{ij})$
  - j-th query of i-th prover where $\beta_{i1}, \ldots \beta_{ij}$ are the messages sent to i-th prover so far



- The following are equivalent:
  - two-prover MIP
  - polynomial-prover MIP
  - Probabilistic Oracle Machine

- we have seen: polynomial-MIP $\subseteq$ Prob. Oracle Machine
- what we need to show: Prob. Oracle Machine $\subseteq$ 2-MIP



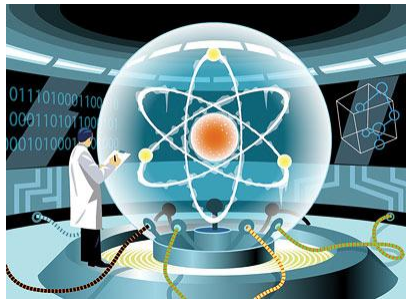
Prob. Oracle Machine ⊆ 2-MIP

Whenever the oracle machine asks the oracle, it instead asks Prover 1



oracle





Prover 1

"But what if Prover 1 try to cheat?"

"Then randomly choose a step and check with the other prover!"

- When someone has claimed an algorithm for EXPTIME-complete problems, how can we check?
  - e.g. generalized chess

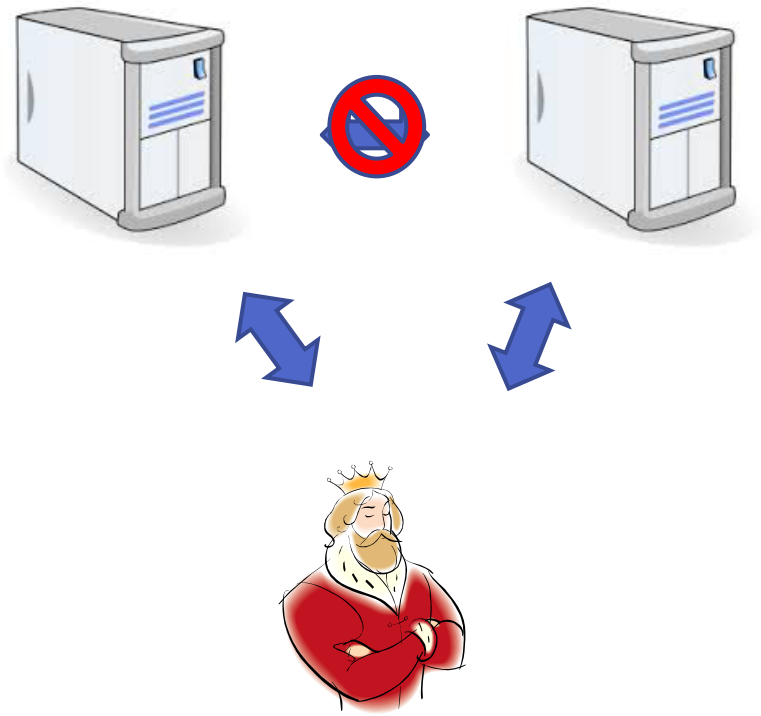


MIP=NEXP=Prob. Oracle machine

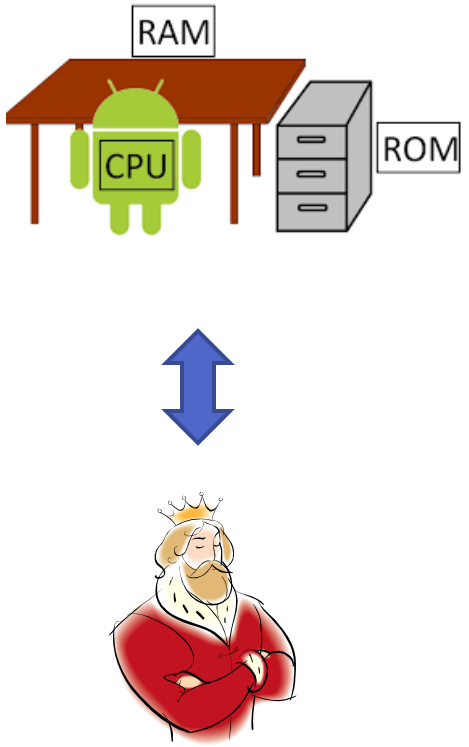


When someone has claimed an algorithm for EXPTIME-complete problems, how can we check?

- Multiprover Interactive proof:



- Prob. Oracle machine:
  - every time reset its memory





MIP*=RE

- Class MIP* of languages that can be decided by a classical verifier interacting with multiple all-powerful quantum provers sharing entanglement is equal to the class of recursively enumerable languages *[Ji, Natarajan, Vidick, Wright, Yuen 2020]*

MIP* = RE

Zhengfeng Ji[*1], Anand Natarajan[†2,3], Thomas Vidick[‡3], John Wright[§2,3,4], and Henry Yuen[¶5]

[1]*Centre for Quantum Software and Information, University of Technology Sydney*
[2]*Institute for Quantum Information and Matter, California Institute of Technology*
[3]*Department of Computing and Mathematical Sciences, California Institute of Technology*
[4]*Department of Computer Science, University of Texas at Austin*
[5]*Department of Computer Science and Department of Mathematics, University of Toronto*

November 7, 2022

Abstract

We show that the class MIP* of languages that can be decided by a classical verifier interacting with multiple all-powerful quantum provers sharing entanglement is equal to the class RE of recursively enumerable languages. Our proof builds upon the quantum low-degree test of (Natarajan and Vidick, FOCS 2018) and the classical low-individual degree test of (Ji, et al., 2020) by integrating recent developments from (Natarajan and Wright, FOCS 2019) and combining them with the recursive compression framework of (Fitzsimons et al., STOC 2019).

An immediate byproduct of our result is that there is an efficient reduction from the Halting Problem to the problem of deciding whether a two-player nonlocal game has entangled value 1 or at most $\frac{1}{2}$. Using a known connection, undecidability of the entangled value implies a negative answer to Tsirelson's problem: we show, by providing an explicit example, that the closure C_{qa} of the set of quantum tensor product correlations is strictly included in the set C_{qc} of quantum commuting correlations. Following work of (Fritz, Rev. Math. Phys. 2012) and (Junge et al., J. Math. Phys. 2011) our results provide a

Decision Trees

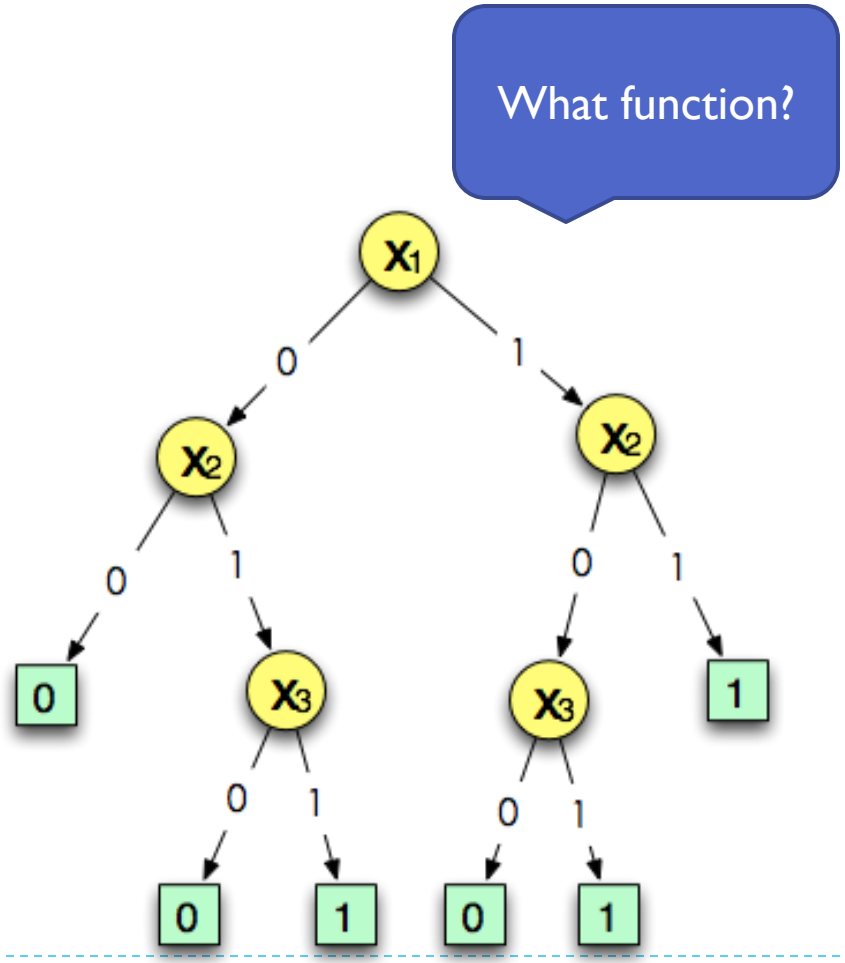
Decision Tree model

- Let $f:\{0,1\}^n \rightarrow \{0,1\}$ be some function
- In a decision tree for f:
  - each node is labeled a variable x_i
  - If $x_i=0$, go left, if $x_i=1$, go right
  - the output at the leaf is f(x)



Decision Tree model

- Let $f:\{0,1\}^n \rightarrow \{0,1\}$ be some function
- In a decision tree for f:
  - each node is labeled a variable x_i
  - If $x_i=0$, go left, if $x_i=1$, go right
  - the output at the leaf is f(x)





Decision tree complexity

- For a tree t and input x, cost(t,x) is the number of bits of x examined by t
- The decision tree complexity of a function f:

$$D(f) = \min_{t \in T_f} \max_{x \in \{0,1\}^n} cost(t,x)$$

where T_f is the set of decision trees that compute f



Examples

- OR function:
  - $f(x_1,x_2,\ldots,x_n)=x_1 \vee x_2 \vee \ldots \vee x_n$

- Lower bound of decision trees?



Examples

- OR function:
  - $f(x_1,x_2,\ldots,x_n)=x_1 \vee x_2 \vee \ldots \vee x_n$

- Lower bound of decision trees:
  - No smaller depth than the trivial bound of n
  - Adversary argument: always respond 0, then the decision tree will be "in suspense" until the last bit



Examples

- Address function: $(n=k+2^k)$

$$f(x_1,\ldots,x_k,y_1,\ldots,y_{2^k}) = y_x$$

  - x is the address for the bit in array y
  - What is the decision tree complexity of f ?



Examples

- Address function: $(n=k+2^k)$

$$f(x_1,\ldots,x_k,y_1,\ldots,y_{2^k}) = y_x$$

  - x is the address for the bit in array y
  - What is the decision tree complexity of f ?

- $D(f) \leq k+1 < \log n+1$
- Also, we can show $D(f)>k$



Certificate complexity

- "Non-deterministic" version of decision tree complexity
- A "0-certificate" for x of a function $f:\{0,1\}^n \to \{0,1\}$:
  - a subset $S \subseteq \{1,2,\ldots,n\}$ s.t. $f(x')=0$ for all $x'|_S = x|_S$
  - So $f(x)=0$

$x|_S$ denotes the substring of x in the coordinates in S



Certificate complexity

- Non-deterministic version of decision tree complexity
- A "0-certificate" for x of a function $f:\{0,1\}^n \to \{0,1\}$:
  - a subset $S \subseteq \{1,2,\ldots,n\}$ s.t. $f(x')=0$ for all $x'|_S = x|_S$
  - So $f(x)=0$
- "1-certificate" is similar such that $f(x')=1$ for all $x'|_S = x|_S$
- So f(x) can be determined by a substring of x



Certificate complexity

- Non-deterministic version of decision tree complexity
- A "0-certificate" for x of a function $f:\{0,1\}^n \to \{0,1\}$:
  - a subset $S \subseteq \{1,2,\ldots,n\}$ s.t. $f(x')=0$ for all $x'|_S = x|_S$
  - So $f(x)=0$

- "1-certificate" is similar such that $f(x')=1$ for all $x'|_S = x|_S$

- Certificate complexity C(f) for a function f:
  - the minimum k s.t. every x has an f(x)-certificate of size ≤k



Example: graph connectivity

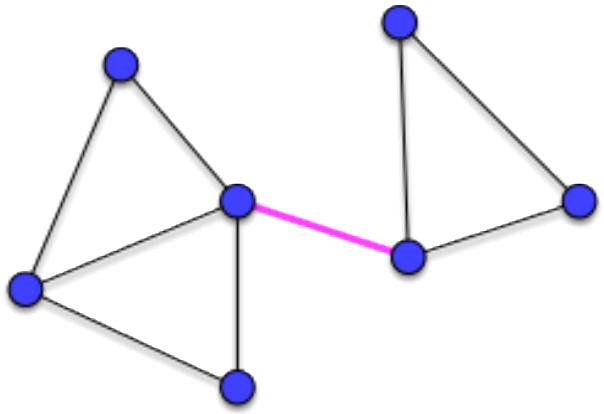
- Graph connectivity function f:
  - Input: an *undirected* graph G with n vertices, represented by a string of length $\binom{n}{2}$ (adjacency matrix)
  - Output: whether the graph is connected

- What about:
  - Decision tree complexity D(f)
  - Certificate complexity C(f)



Example: graph connectivity

- Decision tree complexity D(f)
  - The adversary maintains a "partial" graph which can be extended to connected or disconnected graph using the edges that have not been queried so far
  - If the current edge in remaining subgraph is a "cut-edge", return 1, otherwise return 0



  - So D(f)=n(n-1)/2



Example: graph connectivity

- Graph connectivity function f:
  - Input: an *undirected* graph G with n vertices, represented by a string of length $\binom{n}{2}$ (adjacency matrix)
  - Output: whether the graph is connected

- Decision tree complexity D(f)=n(n-1)/2
- Certificate complexity C(f):
  - 1-certificate: a spanning tree of G



Example: graph connectivity

- Graph connectivity function f:
  - Input: a graph G with n vertices, represented by a string of length $\binom{n}{2}$ (adjacency matrix)
  - Output: whether the graph is connected

- Decision tree complexity D(f)=n(n-1)/2
- Certificate complexity C(f):
  - 1-certificate: a spanning tree of G
  - 0-certificate: a "no-edge" cut
  - So C(f)=max{$n^2/4$,n-1}=$n^2/4$ (if n is even)



- Certificate complexity C and decision tree complexity D:
  - Like NP and P, since NP problem has a "certificate" which can be checked in P
  - What's their relationship?



- Certificate complexity C and decision tree complexity D:
  - Like NP and P, since NP problem has a "certificate" which can be checked in P.
- Trivial: $C(f) \leq D(f)$
- For any function f, $D(f) \leq C(f)^2$

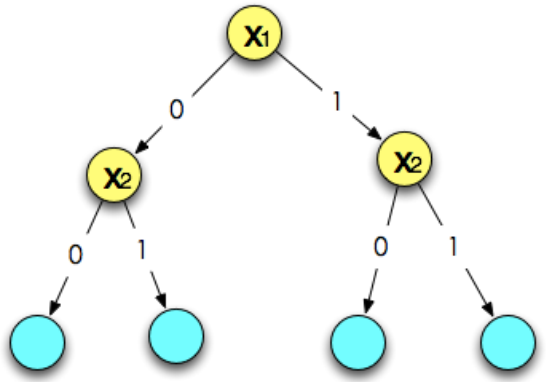


Certificate complexity C and decision tree complexity D:

- For any function f, $D(f) \leq C(f)^2$
  - Proof: Construct the decision tree for f

DT(f)

if f is a constant, return the answer
Find a 0-certificate S for some f(x)=0
Construct the decision tree for all bits in S
For every $y \in \{0,1\}^{|S|}$
call DT($f|_{S=y}$)



recursively construct the decision tree for the function with restriction of x_1 and x_2



Certificate complexity C and decision tree complexity D:

- For any function f, $D(f) \leq C(f)^2$
  - Proof: Construct the decision tree for f

```
DT(f)
        if f is a constant, return the answer
        Find a 0-certificate S for some f(x)=0
        Construct the decision tree for all bits in S
        For every y∈{0,1}^|S|
                call DT(f|_{S=y})
```

- Every 0-certificate must intersect every 1-certificate!
  - Otherwise they will be contradicting to each other



Certificate complexity C and decision tree complexity D:

- For any function f, $D(f) \leq C(f)^2$
  - Proof: Construct the decision tree for f

```
DT(f)
        if f is a constant, return the answer
        Find a 0-certificate S for some f(x)=0
        Construct the decision tree for all bits in S
        For every y∈{0,1}^|S|
                call DT(f|_{S=y})
```

- So every 1-certificate will shrink by 1 after each call
- Thus, in total we need C(f) recursive calls, and the depth of the decision tree is at most $C(f)^2$



Summary

- dIP=NP
- #SAT∈IP
- IP=PSPACE
  - private coins and public coins are equivalent
- MA and AM
  - If Graph Isomorphism∈NPC, then PH collapses to AM
- Discussion of MIP and prob. oracle machine
- Decision tree complexity and certificate complexity

